Information Security (CSNC 3413)

Solved Examples of ALL Syllabus Algorithms: Before Mid and After Mid

Ye PDF aapke course outline ke hisaab se hai: Week 3 se Week 22 tak jitne algorithms aate hain, sab ka formula, ek solved example (step by step) aur exam tip. Har example code se calculate karke standard test vectors se verify kiya gaya hai (DES, AES, RC4, SHA-1, HMAC ke official test values match hui hain).

Syllabus map (course outline ke mutabiq)

Week	Topic (outline)	Section in this PDF
3	Caesar, Monoalphabetic Substitution	1, 4 (Atbash: 3)
4	Vigenère, Auto-key Vigenère	6, 7
5	Vernam Cipher, One-time pad	8, 9
6	Frequency Analysis, Playfair	5, 10
7	Hill Cipher, Extended Euclidean, Cryptanalysis of Caesar	12, 11, 2
8	Rail Fence, Columnar	13, 14
9	Stream cipher: RC4	15
10-11	DES (IP, Feistel, expansion/reduction, key generation, practical question)	16
12-14	AES (encryption, decryption, key scheduling)	17
17-18	RSA	18
19	Diffie-Hellman Key Exchange	19
20-22	Hash functions, SHA, HMAC	20, 21, 22

Pehle ye basics (A-Z numbering, mod)

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

- **A=0, B=1, ... Z=25.** Pehle letters ko numbers me badlo, calculate karo, phir wapas letters.
- **mod 26** = 26 se divide karke remainder. Negative aaye to **+26**, 26 se bada aaye to **-26**.
- **Substitution** = letters badalte hain. **Transposition** = letters ki position badalti hai.
- **Symmetric** = ek hi key (Caesar...AES, RC4). **Asymmetric** = key pair (RSA).

PART A: BEFORE MID TERM (Week 3 to Week 14)

1. Caesar Cipher (Shift / Additive)

Week 3 | Monoalphabetic

Har letter ko **k** positions aage shift karte hain. Slides me ise **Shift cipher** aur **Additive cipher** bhi kaha gaya hai. English ke liye key-space = 26 (practically 25 useful shifts).

FORMULA

Encryption: C = (P + k) mod 26
Decryption: P = (C - k) mod 26

SOLVED EXAMPLE

Q1. Encrypt **MEET ME AFTER THE TOGA PARTY** with k = 17.

```
M(12)+17 = 29 -> 29-26 = 3   -> D
E( 4)+17 = 21                -> V
T(19)+17 = 36 -> 36-26 = 10 -> K
A( 0)+17 = 17                -> R
F( 5)+17 = 22                -> W
R(17)+17 = 34 -> 34-26 = 8  -> I
Cipher: DVVK DV RWKVI KYV KFXR GRIKP
```

Q2 (slide question). Alice aur Bob shift 7 use karte hain. Bob ko mila: **YLTPUN ZHJVYPLZ**. Decrypt karo.

```
P = (C - 7) mod 26
Y(24)-7 = 17 -> R     Z(25)-7 = 18 -> S
L(11)-7 =  4 -> E     H( 7)-7 =  0 -> A
T(19)-7 = 12 -> M     J( 9)-7 =  2 -> C
P(15)-7 =  8 -> I     V(21)-7 = 14 -> O
U(20)-7 = 13 -> N     Y(24)-7 = 17 -> R
N(13)-7 =  6 -> G     P(15)-7 =  8 -> I
                      L(11)-7 =  4 -> E
                      Z(25)-7 = 18 -> S
Result by the formula: REMING SACORIES
```

Note: Formula P = (C - 7) mod 26 se yahi answer aata hai, par ye proper English words nahi hain. Isliye slide ke text me typo ho sakta hai. Exam me method (steps) likho aur apne instructor se ciphertext confirm kar lo.

Trap (k bada ho): k = 29 ho to pehle k mod 26 = 3. XYZ → ABC (X(23)+3 = 26 → 0 = A).

TIP / YAAD RAKHO

k = 13 ko ROT13 kehte hain (do baar apply karo to original wapas).
Key-space chhota hai (26), isliye brute force se toot jata hai (next section).

2. Cryptanalysis of Caesar Cipher

Week 7 | Brute force

Caesar me sirf 25 possible keys hain, isliye attacker saari keys try karke dekh leta hai (**brute-force / exhaustive search**). Jo shift readable plaintext de, wahi sahi key hai.

FORMULA

Har k = 1..25 ke liye: P = (C - k) mod 26, aur dekho kaunsa English me hai.

SOLVED EXAMPLE

Q. Ciphertext **LEELNV LE OLHY** hai. Key nahi pata. Plaintext nikalo.

```
k= 1: KDDKMU KD NKGX
k= 2: JCCJLT JC MJFW
k= 3: IBBIKS IB LIEV
k= 4: HAAHJR HA KHDU
k= 5: GZZGIQ GZ JGCT
k= 6: FYYFHP FY IFBS
k= 7: EXXEGO EX HEAR
k= 8: DWWDFN DW GDZQ
k= 9: CVVCEM CV FCYP
k=10: BUUBDL BU EBXO
k=11: ATTACK AT DAWN   <-- readable English, key = 11
k=12: ZSSZBJ ZS CZVM
k=13: YRRYAI YR BYUL
k=14: XQQXZH XQ AXTK
```

```
k=15: WPPWYG WP ZWSJ
k=16: VOOVXF VO YVRI
k=17: UNNUWE UN XUQH
k=18: TMMTVD TM WTPG
k=19: SLLSUC SL VSOF
k=20: RKKRTB RK URNE
k=21: QJJQSA QJ TQMD
k=22: PIIPRZ PI SPLC
k=23: OHHOQY OH ROKB
k=24: NGGNPX NG QNJA
k=25: MFFMOW MF PMIZ
```

Sirf k = 11 par meaningful text (ATTACK AT DAWN) aaya, to key = 11.

TIP / YAAD RAKHO

Alternative: **frequency analysis**. Sab se common cipher letter ko E maan ke shift nikalo (shift = cipher - E).
Is attack ke liye sirf ciphertext chahiye (ciphertext-only attack).

3. Atbash Cipher (Week 2 slides)

Substitution | Reverse alphabet

Alphabet ko ulta kar do: A↔Z, B↔Y, C↔X. Koi key nahi hoti.

FORMULA

C = (25 - P) mod 26 (A=0 ... Z=25). Decryption wahi formula.

SOLVED EXAMPLE

Q. Encrypt **ATTACKATDAWN**.

```
A(0)->25=Z  T(19)->6=G  T->G  A->Z  C(2)->23=X  K(10)->15=P
A->Z  T->G  D(3)->22=W  A->Z  W(22)->3=D  N(13)->12=M
Cipher: ZGGZXPZGWZDM
Verify: Z(25) -> 25-25 = 0 = A ... original wapas.
```

TIP / YAAD RAKHO

Pehla + aakhri letter ka total hamesha 25 (A+Z, B+Y, M+N). Self-inverse hai.

4. Monoalphabetic Substitution Cipher

Week 3 | Random table

Har plaintext letter ke liye ek fixed random ciphertext letter. Key = poora 26-letter mapping table (permutation).

FORMULA

C = f(P). P = f^{-1}(C). f = permutation of A-Z
Key-space = 26! ≈ 4.03 x 10^{26}

SOLVED EXAMPLE

Q. Cipher table **QWERTYUIOPASDFGHJKLZXCVBNM** (A→Q, B→W, C→E ...). Encrypt **DEFENDTHEEASTWALL**.

```
Plain : A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Cipher: Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

D->R  E->T  F->Y  E->T  N->F  D->R  T->Z  H->I  E->T  E->T
A->Q  S->L  T->Z  W->V  A->Q  L->S  L->S
Cipher: RTYTFRZITTQLZVQSS
```

Dekho E hamesha T banta hai: ye wohi pattern hai jis par **frequency analysis** (next section) kaam karta hai.

TIP / YAAD RAKHO

Key-space bahut bada hai par cipher phir bhi weak hai kyunki letter frequencies hide nahi hoti.

5. Frequency Analysis

Week 6 | Attack on substitution

Har language me letters ki frequency fixed hoti hai. English me sab se common: **E, T, A, O, I, N, S, H, R**. Monoalphabetic cipher me ye frequencies badalti nahi, sirf letter ka naam badalta hai.

FORMULA

Step 1: ciphertext me har letter ki count nikalo.
Step 2: sab se frequent cipher letter → E, phir T, A, O ... guess karo.
Step 3: common words/patterns (THE, AND, double letters EE, TT) se baaki mapping confirm karo.

SOLVED EXAMPLE

Q. Ek monoalphabetic ciphertext (57 letters) hai:
DTTZDTQYZTKZITZGUQHQKZNZITTFTDNLTFZZITOKKTHSNQZZITTCTFOFU
Plaintext guess karo.

```
Letter counts (top):
  T: 14
  Z: 10
  Q: 4
  K: 4
  I: 4
  F: 4

English me sab se common: E, T, A
```

Guess 1: T (14 baar) sab se zyada, to **T = E**.
Guess 2: Z (10 baar) doosre number par, to **Z = T**.
Pattern: cipher me **ZIT** 4 baar aaya. Z = T aur T = E pehle se hain, to ZIT = **T?E**, yaani I = H (THE).
Isi tarah bachi hui letters fill karte jao: plaintext **MEET ME AFTER THE TOGA PARTY THE ENEMY SENT THEIR REPLY AT THE EVENING** mil jata hai.

TIP / YAAD RAKHO

Short ciphertext par frequency analysis weak hoti hai (kam data). Zyada text = zyada reliable.
Digraph (TH, HE, IN, ER) aur trigraph (THE, AND) frequencies bhi use hoti hain.

6. Vigenère Cipher (Polyalphabetic)

Week 4 | Keyword

Plaintext ka har letter alag Caesar cipher se encrypt hota hai. Shift key ke corresponding letter se aati hai. Keyword repeat hota rehta hai. Key-space = 26^n (n = key length).

FORMULA

Encryption: $C_i = (P_i + K_{i \bmod m}) \bmod 26$
Decryption: $P_i = (C_i - K_{i \bmod m}) \bmod 26$ (m = key length)

SOLVED EXAMPLE

Q1. Encrypt **DEFENDTHEEASTWALL** with key **FORTIFICATION**.

```
Plain: D  E  F  E  N  D  T  H  E  E  A  S  T  W  A  L  L
Key  : F  O  R  T  I  F  I  C  A  T  I  O  N  F  O  R  T
(key 13 letters ki hai, 14th se phir F O R T shuru)

D(3) +F(5)  =  8 -> I     E(4) +O(14) = 18 -> S
F(5) +R(17) = 22 -> W     E(4) +T(19) = 23 -> X
N(13)+I(8)  = 21 -> V     D(3) +F(5)  =  8 -> I
T(19)+I(8)  = 27 mod 26 = 1 -> B     H(7)+C(2) = 9 -> J
...
Cipher: ISWXVIBJEXIGGBOCE
```

Q2 (decrypt). Ciphertext **LXFOPVEFRNHR**, key **LEMON**.

```
L(11)-L(11) =  0 -> A     X(23)-E(4)  = 19 -> T
F(5) -M(12) = -7+26 = 19 -> T     O(14)-O(14) = 0 -> A
P(15)-N(13) =  2 -> C     V(21)-L(11) = 10 -> K
E(4) -E(4)  =  0 -> A     F(5) -M(12) = -7+26 = 19 -> T
R(17)-O(14) =  3 -> D     N(13)-N(13) = 0 -> A
H(7) -L(11) = -4+26 = 22 -> W     R(17)-E(4) = 13 -> N
Plain: ATTACKATDAWN
```

DHYAN RAKHO

Decrypt me negative answer aaye to **+26** karna mat bhoolo.

TIP / YAAD RAKHO

Key letter ki value hi shift hai: A=0, B=1, ... F=5. Har letter alag Caesar.

Attack: **Kasiski test** (repeating blocks ki distance se key length) aur Index of Coincidence.

7. Auto-key Vigenère Cipher

Week 4 | Self-extending key

Chhota **primer** (initial keyword) se shuru hota hai, phir **plaintext khud** key stream ban jata hai. Slides ke mutabiq key-space = 26^k (k = primer ki length).

FORMULA

Key stream = primer + plaintext

$C_i = (P_i + K_i) \bmod 26$

$P_i = (C_i - K_i) \bmod 26$ (decrypt karte hue recovered plaintext hi agla key ban jata hai)

SOLVED EXAMPLE

Q1. Encrypt **ATTACKATDAWN**, primer **QUEEN**.

```
Plain : A T T A C K A T D A W N
Key   : Q U E E N A T T A C K A     (QUEEN + ATTACKA)

A(0)+Q(16) = 16 -> Q      C(2) +N(13) = 15 -> P
T(19)+U(20)= 39 mod 26 = 13 -> N   K(10)+A(0)  = 10 -> K
T(19)+E(4) = 23 -> X      A(0)+T(19) = 19 -> T
A(0)+E(4)  =  4 -> E      T(19)+T(19)= 38 mod 26 = 12 -> M
D(3)+A(0)=3->D  A(0)+C(2)=2->C  W(22)+K(10)=32 mod 26=6->G  N(13)+A(0)=13->N
Cipher: QNXEPKTMDCGN
```

Q2 (decrypt). Cipher **QNXEPKTMDCGN**, primer **QUEEN**.

```
Q-Q=0->A  N-U=13-20=-7+26=19->T  X-E=23-4=19->T  E-E=0->A  P-N=15-13=2->C
Ab key = recovered plaintext: A T T A C ...
K-A=10->K  T-T=0->A  M-T=12-19=-7+26=19->T  D-A=3->D  C-C=0->A ...
Plain: ATTACKATDAWN
```

TIP / YAAD RAKHO

Key repeat nahi hoti, islive Kasiski attack Vigenère jaisa kaam nahi karta.

Phir bhi pehle k letters ke baad key = plaintext, isliye known-plaintext / frequency attacks ho sakte hain.

8. Vernam Cipher

Week 5 | Binary XOR stream

Plaintext aur key dono **bits** me hoti hain aur XOR se encrypt hoti hain. Key plaintext ke barabar length ki honi chahiye.

FORMULA

$C = P \oplus K$, $P = C \oplus K$ (XOR: 0⊕0=0, 0⊕1=1, 1⊕0=1, 1⊕1=0)

SOLVED EXAMPLE

Q1. Plaintext **HI**, key **KE** (8-bit ASCII).

```
H = 01001000     I = 01001001
K = 01001011     E = 01000101
XOR  00000011        00001100
Cipher = 00000011 00001100

Decrypt: 00000011 XOR 01001011 = 01001000 = H
         00001100 XOR 01000101 = 01001001 = I
```

Q2. P = 1011 0010, K = 0110 1101 ho to C?

```
1011 0010
0110 1101
---------
1101 1111   = C   (XOR karne par same bits 0, alag bits 1)
```

TIP / YAAD RAKHO

XOR ka rule: **same bits → 0, different bits → 1**.

Same operation encrypt aur decrypt dono karta hai (self-inverse).

9. One-Time Pad (OTP)

Week 5 | Perfect secrecy

Vernam cipher jab **truly random key**, message ke barabar length, aur **sirf ek baar** use ho to One-Time Pad kehlata hai. Ye **unbreakable (perfect secrecy)** hai.

FORMULA

Letters: C = (P + K) mod 26, P = (C - K) mod 26
Bits: C = P ⊕ K

SOLVED EXAMPLE

Q1. Plaintext **HELLO**, pad (key) **XMCKL**.

```
H(7) +X(23) = 30 mod 26 =  4 -> E
E(4) +M(12) = 16 -> Q
L(11)+C(2)  = 13 -> N
L(11)+K(10) = 21 -> V
O(14)+L(11) = 25 -> Z
Cipher: EQNVZ    (decrypt: E(4)-X(23) = -19+26 = 7 = H ...)
```

Q2 (key reuse attack). Agar key K do messages par use ho jaye to C1 ⊕ C2 = (P1 ⊕ K) ⊕ (P2 ⊕ K) = **P1 ⊕ P2**. Key khatam ho jati hai aur attacker ko plaintexts ka relation mil jata hai, isliye key ek hi baar use karte hain.

TIP / YAAD RAKHO

Teen shartein: **random, message jitni lambi, ek hi baar use**.
Problem: itni lambi key ko securely share karna mushkil hai.

10. Playfair Cipher (Digram)

Week 6 | 5x5 grid

Pehla digram/bigram substitution cipher. Plaintext ko **2-2 letters** me todte hain aur 5x5 grid ke rules se encrypt karte hain. I/J ek hi cell.

FORMULA

Same row: har letter ko **right** wale se replace (wrap-around).
Same column: har letter ko **neeche** wale se replace (wrap-around).
Alag row aur column: rectangle banao, **opposite corner** wale letters lo (apni row me).
Digram me same letter ho to beech me **filler** daalo (slide me Q). Odd length par end me filler.
Decryption: left / up / rectangle (same).

SOLVED EXAMPLE

Q (slide example). Keyword **ENVIRONMENT**, message **WRAPPINGOOP**, filler **Q**.

```
Keyword (repeat hata ke): E N V I R O M T   +  baaki alphabet (I/J ek)

Grid:
            E N V I R
            O M T A B
            C D F G H
            K L P Q S
            U W X Y Z

Digrams: WR | AP | PI | NG | OO -> OQ | OP     (OO double letter, beech me Q)

WR: rectangle   -> ZN
AP: rectangle   -> TQ
PI: rectangle   -> QV
NG: rectangle   -> ID
OQ: rectangle   -> AK
OP: rectangle   -> TK

Cipher: ZN TQ QV ID AK TK   (slide me I/J cell ko 'J' likha hai, to ZNTQQVJDAKTK)
```

Decrypt check (left / up / rectangle):

```
ZN: rectangle   -> WR
TQ: rectangle   -> AP
QV: rectangle   -> PI
ID: rectangle   -> NG
AK: rectangle   -> OQ
TK: rectangle   -> OP
```

```
Plain digrams: WR AP PI NG OQ OP  -> WRAPPINGOQOP, filler Q hata ke WRAPPINGOOP
```

DHYAN RAKHO

Grid banate waqt keyword ke repeat letters (ENVIRONMENT me N, O, R, ...) sirf pehli baar likhe jate hain.

TIP / YAAD RAKHO

Pehle sab digrams likho, phir har digram par rule lagao.
Rectangle me **apni row** rakho aur doosre letter ka **column** lo.
Key-space 25! ≈ 1.55×10^{25} (maximum, full permutation).
Slide me filler Q hai, isliye exam me wohi filler use karo jo question/instructor de.

11. Extended Euclidean Algorithm

Week 7 | Modular inverse

gcd(a, n) nikalta hai aur saath me **s, t** bhi deta hai jin se **a·s + n·t = gcd**. Agar gcd = 1 ho to $\mathbf{a^{-1} \bmod n = t \bmod n}$. Hill cipher, Affine aur RSA (d nikalne) me ye use hota hai.

FORMULA

$r_i = r_{i-2} - q_i \cdot r_{i-1}$, $s_i = s_{i-2} - q_i \cdot s_{i-1}$, $t_i = t_{i-2} - q_i \cdot t_{i-1}$
Start: (r, s, t) = (n, 1, 0) aur (a, 0, 1). Jab r = 0 ho, pichli row ka r = gcd.

SOLVED EXAMPLE

Q1. 15 ka inverse mod 26 nikalo.

```
Rows (n=26, a=15):
  q |    r |    s |    t
    |   26 |    1 |    0
    |   15 |    0 |    1
  1 |   11 |    1 |   -1
  1 |    4 |   -1 |    2
  2 |    3 |    3 |   -5
  1 |    1 |   -4 |    7
  3 |    0 |   15 |  -26
gcd = 1.  26(-4) + 15(7) = -104 + 105 = 1
15^-1 mod 26 = t = 7   (check: 15 x 7 = 105 = 4x26 + 1)
```

Q2 (RSA wala). 17 ka inverse mod 3120.

```
  q |    r |    s |    t
    | 3120 |    1 |    0
    |   17 |    0 |    1
183 |    9 |    1 | -183
  1 |    8 |   -1 |  184
  1 |    1 |    2 | -367
  8 |    0 |  -17 | 3120
gcd = 1.  3120(2) + 17(-367) = 6240 - 6239 = 1
t = -367  ->  -367 + 3120 = 2753
17^-1 mod 3120 = 2753   (check: 17 x 2753 = 46801 = 15 x 3120 + 1)
```

TIP / YAAD RAKHO

Table me last se pehli row (jaha agli r = 0 hoti hai) ka t inverse hota hai.
t negative ho to n add karo.

12. Hill Cipher

Week 7 | Matrix

n letters ka block ek vector P hota hai, jise n x n **key matrix K** se multiply karte hain (mod 26).

FORMULA

Encryption: $C = K \cdot P \bmod 26$
Decryption: $P = K^{-1} \cdot C \bmod 26$. $K^{-1} = \det(K)^{-1} \cdot \text{adj}(K) \bmod 26$
Condition: gcd(det K, 26) = 1.

SOLVED EXAMPLE

Q1 (2x2). K = [[3, 3], [2, 5]]. Encrypt **HELP** aur decrypt karo.

```
HE = [7, 4]   :  [3*7 + 3*4,  2*7 + 5*4]  = [33, 34] mod 26 = [7, 8]   -> HI
LP = [11,15] :  [3*11+3*15, 2*11+5*15]  = [78, 97] mod 26 = [0, 19]  -> AT
Cipher: HIAT
Inverse: det = 3*5 - 3*2 = 9;  9^-1 mod 26 = 3  (9*3 = 27 = 1)
```

```
adj(K) = [[5, -3], [-2, 3]]   (diagonal swap, off-diagonal sign badlo)
K^-1 = 3 * adj = [[15, -9], [-6, 9]] mod 26 = [[15, 17], [20, 9]]

Decrypt HI = [7, 8]: [15*7+17*8, 20*7+9*8] = [241, 212] mod 26 = [7, 4] -> HE
Decrypt AT = [0,19]: [15*0+17*19, 20*0+9*19] = [323, 171] mod 26 = [11,15] -> LP
Plain: HELP
```

Q2 (3x3). K = [[6, 24, 1], [13, 16, 10], [20, 17, 15]] (GYBNQKURP). Encrypt **ACT**.

```
P = [0, 2, 19]
Row1: 6*0 + 24*2 + 1*19   =  67 mod 26 = 15 -> P
Row2: 13*0 + 16*2 + 10*19 = 222 mod 26 = 14 -> O
Row3: 20*0 + 17*2 + 15*19 = 319 mod 26 =  7 -> H
Cipher: POH

det(K) = 441;  441 mod 26 = 25;  gcd(25,26)=1 valid;  25^-1 = 25
K^-1 mod 26 = [[8, 5, 10], [21, 8, 21], [21, 12, 8]]
Check: K^-1 * [15,14,7]: Row1 = 8*15+5*14+10*7 = 260 = 10*26 -> 0 = A  (original)
```

DHYAN RAKHO

det(K) 26 ke saath coprime na ho (even ya 13 ka multiple) to decrypt nahi ho sakta.

TIP / YAAD RAKHO

2x2 inverse shortcut: diagonal swap, off-diagonal ka sign badlo, det inverse se multiply, mod 26.
Hill ek linear cipher hai: known-plaintext attack se toot jata hai.

13. Rail Fence Cipher

Week 8 | Transposition

Plaintext ko **zig-zag** (upar-neeche) rails par likh kar, **rail by rail** row-wise padhte hain.

FORMULA

Rails ki sequence (depth 3): 1, 2, 3, 2, 1, 2, 3, 2 ... cycle = 2(depth - 1)
Decrypt: har rail ki length gino, cipher ko rails me bharo, zig-zag me padho.

SOLVED EXAMPLE

Q1 (slide). MEET ME AFTER THE TOGA PARTY, depth 3.

```
M . . . M . . . T . . . H . . . G . . . R . .
. E . T . E . F . E . T . E . O . A . A . T .
. . E . . . A . . . R . . . T . . . P . . . Y

Row 1: MMTHGR
Row 2: ETEFETEOAAT
Row 3: EARTPY
Cipher: MMTHGR ETEFETEOAAT EARTPY
```

Q2 (depth 4, encrypt + decrypt). DEFENDTHEEASTWALLNOW, depth 4.

```
D . . . . . T . . . . . T . . . . . O .
. E . . . D . H . . . S . W . . . N . W
. . F . N . . . E . A . . . A . L . . .
. . . E . . . . . E . . . . . L . . . .

Row 1: DTTO
Row 2: EDHSWNW
Row 3: FNEAAL
Row 4: EEL
Cipher: DTTOEDHSWNWFNEAALEEL

Decrypt: rail lengths = [4, 7, 6, 3].
Cipher ke pehle 4 letters rail 1, agle 7 rail 2, agle 6 rail 3, baaki rail 4.
Zig-zag me wapas padho -> DEFENDTHEEASTWALLNOW
```

TIP / YAAD RAKHO

Key sirf depth hai, islive brute force aasaan hai.
Decrypt ke liye pehle zig-zag pattern (rail numbers) likho, phir letters bharo.

14. Columnar Transposition Cipher

Week 8 | Keyword columns

Plaintext ko keyword ki width ke rows me likhte hain. Keyword ke letters ko alphabet order me number do aur **columns ko usi order me** upar se neeche padho.

FORMULA

Columns = len(key). Rows = ceil(len(text) / columns) (kam ho to X padding)
Cipher = columns ko key ke number order me padhna

SOLVED EXAMPLE

Q. ATTACKPOSTPONEDUNTILTOMORROW (28 letters), key **CIPHER**. Encrypt aur decrypt karo.

```
Key    : C  I  P  H  E  R
Number : 1  4  5  3  2  6     (C=1, E=2, H=3, I=4, P=5, R=6)
         A  T  T  A  C  K
         P  O  S  T  P  O
         N  E  D  U  N  T
         I  L  T  O  M  O
         R  R  O  W  X  X     (last row me XX padding)

Col 1 (key C): APNIR
Col 2 (key E): CPNMX
Col 3 (key H): ATUOW
Col 4 (key I): TOELR
Col 5 (key P): TSDTO
Col 6 (key R): KOTOX
Cipher: APNIR CPNMX ATUOW TOELR TSDTO KOTOX

Decrypt: 30 letters / 6 columns = 5 rows. Cipher ko 5-5 ke blocks me todo;
pehla block Col 1 (C), doosra Col 2 (E), ... Phir columns ko key order (C I P H E R)
me wapas rakh ke row-wise padho:
  ATTACK
  POSTPO
  NEDUNT
  ILTOMO
  RROWXX
-> ATTACKPOSTPONEDUNTILTOMORROWXX (XX hata do)
```

DHYAN RAKHO

Column order alphabet ke hisaab se hota hai, key ke left-to-right nahi.

TIP / YAAD RAKHO

Keyword me repeat letters ho to left-to-right unko number do.
Transposition me letter frequencies same rehti hain (sirf position badalti hai).

15. RC4 (Stream Cipher)

Week 9 | Key stream

RC4 ek byte-oriented **stream cipher** hai: S-array (0..255) ko key se scramble karta hai (KSA), phir keystream bytes banata hai (PRGA) aur plaintext se XOR karta hai. Neeche N = 8 ka chhota example hai (exam style); asli RC4 me N = 256 aur mod 256.

FORMULA

KSA: S[i] = i; j = 0; for i = 0 to N-1: j = (j + S[i] + K[i mod keylen]) mod N; swap(S[i], S[j])
PRGA: i = (i+1) mod N; j = (j + S[i]) mod N; swap(S[i], S[j]); t = (S[i] + S[j]) mod N; keystream k = S[t]
Encrypt/Decrypt: C = P ⊕ k, P = C ⊕ k

SOLVED EXAMPLE

Q. N = 8, key K = [1, 2, 3, 6], plaintext = [1, 2, 2, 2]. Ciphertext nikalo.

```
S initially = [0, 1, 2, 3, 4, 5, 6, 7]    j = 0

KSA:
i | j = (j + S[i] + K[i mod 4]) mod 8 |  S after swap
0 | j = (0 + 0 + 1) mod 8 = 1           | [1, 0, 2, 3, 4, 5, 6, 7]
1 | j = (1 + 0 + 2) mod 8 = 3           | [1, 3, 2, 0, 4, 5, 6, 7]
2 | j = (3 + 2 + 3) mod 8 = 0           | [2, 3, 1, 0, 4, 5, 6, 7]
3 | j = (0 + 0 + 6) mod 8 = 6           | [2, 3, 1, 6, 4, 5, 0, 7]
4 | j = (6 + 4 + 1) mod 8 = 3           | [2, 3, 1, 4, 6, 5, 0, 7]
5 | j = (3 + 5 + 2) mod 8 = 2           | [2, 3, 5, 4, 6, 1, 0, 7]
6 | j = (2 + 0 + 3) mod 8 = 5           | [2, 3, 5, 4, 6, 0, 1, 7]
7 | j = (5 + 7 + 6) mod 8 = 2           | [2, 3, 7, 4, 6, 0, 1, 5]
```

```
S after KSA = [2, 3, 7, 4, 6, 0, 1, 5]

PRGA (i = 0, j = 0 se start):
i | j = (j + S[i]) mod 8 | t = (S[i]+S[j]) mod 8 | keystream S[t] | S
1 | j = 3                | t = 7                 | k = 5            | [2, 4, 7, 3, 6, 0, 1, 5]
2 | j = 2                | t = 6                 | k = 1            | [2, 4, 7, 3, 6, 0, 1, 5]
3 | j = 5                | t = 3                 | k = 0            | [2, 4, 7, 0, 6, 3, 1, 5]
4 | j = 3                | t = 6                 | k = 1            | [2, 4, 7, 6, 0, 3, 1, 5]

Keystream = [5, 1, 0, 1]
Plain     = [1, 2, 2, 2]
Cipher    = plain XOR keystream = [4, 3, 2, 3]
  (1^5=4, 2^1=3, 2^0=2, 2^1=3)
Decrypt: cipher XOR keystream = [1, 2, 2, 2]  (original)
```

Real RC4 test (N = 256): key = "Key", plaintext = "Plaintext" → ciphertext = **BBF316E8D940AF0AD3** (hex).

TIP / YAAD RAKHO

KSA ek baar chalta hai, PRGA har byte ke liye.
Mod hamesha N ka (8 example me, asli me 256).
Same key dobara use na karo (stream cipher me key reuse khatarnak hai).

16. DES (Data Encryption Standard)

Week 10-11 | Feistel block cipher

DES **64-bit block** ko **56-bit key** (64 bit with 8 parity bits) se encrypt karta hai. Structure: **Initial Permutation (IP)**, **16 Feistel rounds**, aakhir me left/right swap aur **Final Permutation (FP = IP^{-1})**. Har round me 48-bit subkey K_i use hoti hai. Claude Shannon ke concepts: **confusion** (S-boxes: key aur ciphertext ka relation chhupana) aur **diffusion** (P-box/permutation: ek bit ka asar poore block me failana).

FORMULA

Feistel round: $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

$F(R, K) = P(S(E(R) \oplus K))$: E: 32 → 48 bit (expansion), S: 8 S-boxes (6 bit → 4 bit), P: 32-bit permutation

S-box lookup: 6-bit input b1 b2 b3 b4 b5 b6 me **row = b1b6** (0-3), **column = b2b3b4b5** (0-15).

Key schedule: 64-bit key → PC-1 (56 bit) → C0, D0 (28-28) → har round left shift (1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1) → PC-2 (48 bit K_i).

Decryption = same algorithm, bas subkeys ulte order me (K16 ... K1).

SOLVED EXAMPLE

Practical question. Key K = **133457799BBCDFF1**, Plaintext M = **0123456789ABCDEF** (hex). DES encryption karo.

Step 1: Key generation.

```
K  = 00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001
K+ = PC-1(K) = 1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111
C0 = 1111 0000 1100 1100 1010 1010 1111
D0 = 0101 0101 0110 0110 0111 1000 1111

Left shifts se C_i, D_i:
C0  = 1111 0000 1100 1100 1010 1010 1111  D0  = 0101 0101 0110 0110 0111 1000 1111
C1  = 1110 0001 1001 1001 0101 0101 1111  D1  = 1010 1010 1100 1100 1111 0001 1110   (shift 1)
C2  = 1100 0011 0011 0010 1010 1011 1111  D2  = 0101 0101 1001 1001 1110 0011 1101   (shift 1)
C3  = 0000 1100 1100 1010 1010 1111 1111  D3  = 0101 0110 0110 0111 1000 1111 0101   (shift 2)
C4  = 0011 0011 0010 1010 1011 1111 1100  D4  = 0101 1001 1001 1110 0011 1101 0101   (shift 2)
C5  = 1100 1100 1010 1010 1111 1111 0000  D5  = 0110 0110 0111 1000 1111 0101 0101   (shift 2)
C6  = 0011 0010 1010 1011 1111 1100 0011  D6  = 1001 1001 1110 0011 1101 0101 0101   (shift 2)
C7  = 1100 1010 1010 1111 1111 0000 1100  D7  = 0110 0111 1000 1111 0101 0101 0110   (shift 2)
C8  = 0010 1010 1011 1111 1100 0011 0011  D8  = 1001 1110 0011 1101 0101 0101 1001   (shift 2)
C9  = 0101 0101 0111 1111 1000 0110 0110  D9  = 0011 1100 0111 1010 1010 1011 0011   (shift 1)
C10 = 0101 0101 1111 1110 0001 1001 1001  D10 = 1111 0001 1110 1010 1010 1100 1100   (shift 2)
C11 = 0101 0111 1111 1000 0110 0110 0101  D11 = 1100 0111 1010 1010 1011 0011 0011   (shift 2)
C12 = 0101 1111 1110 0001 1001 1001 0101  D12 = 0001 1110 1010 1010 1100 1100 1111   (shift 2)
C13 = 0111 1111 1000 0110 0110 0101 0101  D13 = 0111 1010 1010 1011 0011 0011 1100   (shift 2)
C14 = 1111 1110 0001 1001 1001 0101 0101  D14 = 1110 1010 1010 1100 1100 1111 0001   (shift 2)
C15 = 1111 1000 0110 0110 0101 0101 0111  D15 = 1010 1010 1011 0011 0011 1100 0111   (shift 2)
C16 = 1111 0000 1100 1100 1010 1010 1111  D16 = 0101 0101 0110 0110 0111 1000 1111   (shift 1)

K_i = PC-2(C_i D_i):
K1  = 000110 110000 001011 101111 111111 000111 000001 110010
K2  = 011110 011010 111011 011001 110110 111100 100111 100101
K3  = 010101 011111 110010 001010 010000 101100 111110 011001
K4  = 011100 101010 110111 010110 110110 110011 010100 011101
K5  = 011111 001110 110000 000111 111010 110101 001110 101000
K6  = 011000 111010 010100 111110 010100 000111 101100 101111
K7  = 111011 001000 010010 110111 111101 100001 100010 111100
K8  = 111101 111000 101000 111010 110000 010011 101111 111011
K9  = 111000 001101 101111 101011 111011 011110 011110 000001
K10 = 101100 011111 001101 000111 101110 100100 011001 001111
K11 = 001000 010101 111111 010011 110111 101101 001110 000110
K12 = 011101 010111 000111 110101 100101 000110 011111 101001
K13 = 100101 111100 010111 010001 111110 101011 101001 000001
K14 = 010111 110100 001110 110111 111100 101110 011100 111010
K15 = 101111 111001 000110 001101 001111 010011 111100 001010
K16 = 110010 110011 110110 001011 000011 100001 011111 110101
```

Step 2: Initial Permutation.

```
M     = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111
IP(M) = 1100 1100 0000 0000 1100 1100 1111 1111 1111 0000 1010 1010 1111 0000 1010 1010
L0 = 1100 1100 0000 0000 1100 1100 1111 1111   (CC00CCFF)
R0 = 1111 0000 1010 1010 1111 0000 1010 1010   (F0AAF0AA)
```

Step 3: Round 1 (L1 = R0 and R1 = L0 ⊕ F(R0, K1)).

```
E(R0)      = 011110 100001 010101 010101 011110 100001 010101 010101
```

```
K1         = 000110 110000 001011 101111 111111 000111 000001 110010
E(R0) xor K1 = 011000 010001 011110 111010 100001 100110 010100 100111

S-box |  B (6 bit) | row=b1b6 | col=b2b3b4b5 | output
  S1  |  011000   |  0 (00)   |  12 (1100)    |  5 = 0101
  S2  |  010001   |  1 (01)   |   8 (1000)    | 12 = 1100
  S3  |  011110   |  0 (00)   |  15 (1111)    |  8 = 1000
  S4  |  111010   |  2 (10)   |  13 (1101)    |  2 = 0010
  S5  |  100001   |  3 (11)   |   0 (0000)    | 11 = 1011
  S6  |  100110   |  2 (10)   |   3 (0011)    |  5 = 0101
  S7  |  010100   |  0 (00)   |  10 (1010)    |  9 = 1001
  S8  |  100111   |  3 (11)   |   3 (0011)    |  7 = 0111

S-box outputs = 0101 1100 1000 0010 1011 0101 1001 0111
P(outputs) = F = 0010 0011 0100 1010 1010 1001 1011 1011   (234AA9BB)

R1 = L0 xor F = 1110 1111 0100 1010 0110 0101 0100 0100   (EF4A6544)
L1 = R0       = 1111 0000 1010 1010 1111 0000 1010 1010   (F0AAF0AA)
```

Step 4: Baaki 15 rounds isi tarah. Har round ke baad (L, R) hex me:

```
Round 0   L = CC00CCFF   R = F0AAF0AA
Round 1   L = F0AAF0AA   R = EF4A6544
Round 2   L = EF4A6544   R = CC017709
Round 3   L = CC017709   R = A25C0BF4
Round 4   L = A25C0BF4   R = 77220045
Round 5   L = 77220045   R = 8A4FA637
Round 6   L = 8A4FA637   R = E967CD69
Round 7   L = E967CD69   R = 064ABA10
Round 8   L = 064ABA10   R = D5694B90
Round 9   L = D5694B90   R = 247CC67A
Round 10  L = 247CC67A   R = B7D5D7B2
Round 11  L = B7D5D7B2   R = C5783C78
Round 12  L = C5783C78   R = 75BD1858
Round 13  L = 75BD1858   R = 18C3155A
Round 14  L = 18C3155A   R = C28C960D
Round 15  L = C28C960D   R = 43423234
Round 16  L = 43423234   R = 0A4CD995
```

Step 5: Final swap aur FP.

```
R16 L16 = 0000 1010 0100 1100 1101 1001 1001 0101 0100 0011 0100 0010 0011 0010 0011 0100
FP(R16 L16) = 1000 0101 1110 1000 0001 0011 0101 0100 0000 1111 0000 1010 1011 0100 0000 0101
Ciphertext  = 85E813540F0AB405
Decrypt check: DES-decrypt(85E813540F0AB405, K) = 0123456789ABCDEF  (original M)
```

DHYAN RAKHO

Last round ke baad L16 aur R16 **swap** hote hain (R16 L16), phir FP lagta hai.
Tables (IP, E, P, PC-1, PC-2, S-boxes) is PDF ke end me Appendix me di gayi hain.

TIP / YAAD RAKHO

Exam practical me aksar **sirf Round 1** poocha jata hai: key generation (K1), IP, E(R0), XOR, S-box lookup, P, R1.
Step ka order yaad rakho: **E → XOR K → S-box → P → XOR L**.
S-box me row = pehla + aakhri bit, column = beech ke 4 bits.

17. AES (Advanced Encryption Standard)

AES **128-bit block** ko 128/192/256-bit key se encrypt karta hai. Ye **SPN (substitution-permutation network)** hai, Feistel nahi. Data ko **4x4 byte state** (column-major) me rakhte hain. AES-128: **10 rounds**, 11 round keys.

FORMULA

Kev size / rounds: 128 bit → 10. 192 bit → 12. 256 bit → 14

Initial: AddRoundKey. **Rounds 1-9:** SubBytes → ShiftRows → MixColumns → AddRoundKey. **Round 10:** SubBytes → ShiftRows → AddRoundKey (**MixColumns nahi**).

SubBytes: har byte S-box se replace (high nibble = row, low nibble = column). **ShiftRows:** row r ko r bytes left rotate (row 0 same).

MixColumns: har column ko fixed matrix se GF(2^8) me multiply. **AddRoundKey:** state ⊕ round key.

Key schedule (AES-128): $w_i = w_{i-4} \oplus w_{i-1}$; agar i mod 4 = 0 to $w_i = w_{i-4} \oplus g(w_{i-1})$, g = RotWord, SubWord, ⊕ Rcon.

Decryption: inverse operations ulte order me: InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns (MixColumns inverse matrix 0e 0b 0d 09).

SOLVED EXAMPLE

Q. Plaintext = **3243f6a8885a308d313198a2e0370734**, Key = **2b7e151628aed2a6abf7158809cf4f3c** (FIPS-197 Appendix B). AES-128 encrypt karo.

Step 1: Key expansion.

```
w0..w3 = 2b7e1516 28aed2a6 abf71588 09cf4f3c   (original key)

w4 = w0 xor g(w3):
  RotWord(w3) = cf4f3c09
  SubWord     = 8a84eb01
  xor Rcon[1] = 8b84eb01   (Rcon[1] = 01 00 00 00)
  w4 = w0 xor that = 2b7e1516 xor 8b84eb01 = a0fafe17
w5 = w4 xor w1 = 88542cb1
w6 = w5 xor w2 = 23a33939
w7 = w6 xor w3 = 2a6c7605

All round keys:
Round 0 : 2b7e1516 28aed2a6 abf71588 09cf4f3c
Round 1 : a0fafe17 88542cb1 23a33939 2a6c7605
Round 2 : f2c295f2 7a96b943 5935807a 7359f67f
Round 3 : 3d80477d 4716fe3e 1e237e44 6d7a883b
Round 4 : ef44a541 a8525b7f b671253b db0bad00
Round 5 : d4d1c6f8 7c839d87 caf2b8bc 11f915bc
Round 6 : 6d88a37a 110b3efd dbf98641 ca0093fd
Round 7 : 4e54f70e 5f5fc9f3 84a64fb2 4ea6dc4f
Round 8 : ead27321 b58dbad2 312bf560 7f8d292f
Round 9 : ac7766f3 19fadc21 28d12941 575c006e
Round 10: d014f9a8 c9ee2589 e13f0cc8 b6630ca6
```

Step 2: State aur Round 0 (AddRoundKey).

```
Plaintext      Round key 0    State (xor)
32 88 31 e0    2b 28 ab 09    19 a0 9a e9
43 5a 31 37    7e ae f7 cf    3d f4 c6 f8
f6 30 98 07    15 d2 15 4f    e3 e2 8d 48
a8 8d a2 34    16 a6 88 3c    be 2b 2a 08
(column-major: pehle 4 bytes pehla column)
```

Step 3: Round 1.

```
Start          SubBytes       ShiftRows      MixColumns
19 a0 9a e9    d4 e0 b8 1e    d4 e0 b8 1e    04 e0 48 28
3d f4 c6 f8    27 bf b4 41    bf b4 41 27    66 cb f8 06
e3 e2 8d 48    11 98 5d 52    5d 52 11 98    81 19 d3 26
be 2b 2a 08    ae f1 e5 30    30 ae f1 e5    e5 9a 7a 4c

MixColumns     Round key 1    End of round 1
04 e0 48 28    a0 88 23 2a    a4 68 6b 02
66 cb f8 06    fa 54 a3 6c    9c 9f 5b 6a
81 19 d3 26    fe 2c 39 76    7f 35 ea 50
e5 9a 7a 4c    17 b1 39 05    f2 2b 43 49
```

SubBytes example: byte **19** → row 1, column 9 of S-box → **d4**. **ShiftRows:** row 1 ek position left, row 2 do positions, row 3 teen positions (row 0 same).

MixColumns detail (pehle byte ke liye):

```
Column 0 (ShiftRows ke baad) = d4 bf 5d 30
MixColumns matrix:   [02 03 01 01]   [s0]
                     [01 02 03 01] x [s1]   (GF(2^8), irreducible poly x^8+x^4+x^3+x+1 = 0x11b)
                     [01 01 02 03]   [s2]
                     [03 01 01 02]   [s3]

s0' = 02*d4 xor 03*bf xor 5d xor 30
  02*d4: d4 << 1 = 1a8 (MSB set, to 0x1b se xor) -> b3
  03*bf = 02*bf xor bf = 65 xor bf = da
  s0' = b3 xor da xor 5d xor 30 = 04
Isi tarah baaki bytes: column 0 output = 04 66 81 e5
```

Step 4: Rounds 2 se 10 same tarah (round 10 me MixColumns nahi). Har round ke baad state:

```
After round 1 : a4 9c 7f f2 68 9f 35 2b 6b 5b ea 43 02 6a 50 49
After round 2 : aa 8f 5f 03 61 dd e3 ef 82 d2 4a d2 68 32 46 9a
After round 3 : 48 6c 4e ee 67 1d 9d 0d 4d e3 b1 38 d6 5f 58 e7
After round 4 : e0 92 7f e8 c8 63 63 c0 d9 b1 35 50 85 b8 be 01
After round 5 : f1 00 6f 55 c1 92 4c ef 7c c8 8b 32 5d b5 d5 0c
After round 6 : 26 0e 2e 17 3d 41 b7 7d e8 64 72 a9 fd d2 8b 25
After round 7 : 5a 41 42 b1 19 49 dc 1f a3 e0 19 65 7a 8c 04 0c
After round 8 : ea 83 5c f0 04 45 33 2d 65 5d 98 ad 85 96 b0 c5
After round 9 : eb 40 f2 1e 59 2e 38 84 8b a1 13 e7 1b c3 42 d2
After round 10: 39 25 84 1d 02 dc 09 fb dc 11 85 97 19 6a 0b 32

Ciphertext = 3925841d02dc09fbdc118597196a0b32
```

Decrypt check: inverse rounds lagane par ciphertext se wapas **3243f6a8885a308d313198a2e0370734** milta hai (original plaintext).

DHYAN RAKHO

ShiftRows row 0 ko nahi hilata: row 1 ko 1, row 2 ko 2, row 3 ko 3 positions left.
State column-major bharta hai (row-wise nahi): input bytes b0 b1 b2 b3 pehla column.

TIP / YAAD RAKHO

Yaad rakho: **Sub. Shift. Mix. Add** (SSMA). Aakhri round me Mix nahi.
Key schedule: har 4th word par g() (Rot, Sub, Rcon), baaki simple XOR.
Exam me S-box table mostly di jati hai (Appendix me bhi hai).

PART B: AFTER MID TERM (Week 17 to Week 22)

18. RSA Algorithm

Week 17-18 | Public key

Asymmetric (public key) cryptosystem. Security is baat par hai ki do bade primes ka product **factor karna mushkil** hai. Public key = (e, n), private key = (d, n).

FORMULA

1) Do primes p, q chuno. 2) $n = p \cdot q$. 3) $\varphi(n) = (p - 1)(q - 1)$.
4) e chuno: $1 < e < \varphi(n)$ aur $\gcd(e, \varphi(n)) = 1$.
5) $d = e^{-1} \bmod \varphi(n)$. yaani $\mathbf{e \cdot d \bmod \varphi(n) = 1}$ (Extended Euclid).
Encrypt: $C = M^e \bmod n$ **Decrypt:** $M = C^d \bmod n$ $(M < n)$
Digital signature: $S = M^d \bmod n$ (private key se), verify: $M = S^e \bmod n$ (public key se).

SOLVED EXAMPLE

Q1 (chhota textbook). p = 3, q = 11, e = 7, M = 5.

```
n = 3 x 11 = 33,   phi = 2 x 10 = 20
e = 7: gcd(7, 20) = 1 OK.   d: 7 x d = 1 mod 20 -> 7 x 3 = 21 = 1 mod 20 -> d = 3
Public key (7, 33),  Private key (3, 33)

Encrypt: C = 5^7 mod 33 = 78125 mod 33 = 14
Decrypt: M = 14^3 mod 33 = 2744 mod 33 = 5   (original)
```

Q2. p = 11, q = 13, e = 7, M = 9 (square-and-multiply se).

```
n = 143, phi = 10 x 12 = 120,  e = 7 (gcd(7,120)=1)
d: 7 x d = 1 mod 120 -> 7 x 103 = 721 = 6 x 120 + 1 -> d = 103

Encrypt: C = 9^7 mod 143
Exponent 7 = 111 (binary)
bit | square           | multiply by 9 (agar bit=1) | result
 1  | 1^2 mod 143 = 1      | 1*9 mod 143 = 9      | 9
 1  | 9^2 mod 143 = 81     | 81*9 mod 143 = 14     | 14
 1  | 14^2 mod 143 = 53     | 53*9 mod 143 = 48     | 48
C = 48

Decrypt: M = 48^103 mod 143
Exponent 103 = 1100111 (binary)
bit | square           | multiply by 48 (agar bit=1) | result
 1  | 1^2 mod 143 = 1      | 1*48 mod 143 = 48     | 48
 1  | 48^2 mod 143 = 16     | 16*48 mod 143 = 53     | 53
 0  | 53^2 mod 143 = 92     | -                     | 92
 0  | 92^2 mod 143 = 27     | -                     | 27
 1  | 27^2 mod 143 = 14     | 14*48 mod 143 = 100    | 100
 1  | 100^2 mod 143 = 133    | 133*48 mod 143 = 92     | 92
 1  | 92^2 mod 143 = 27     | 27*48 mod 143 = 9      | 9
M = 9   (original)
```

Q3 (bada). p = 61, q = 53, e = 17, M = 65.

```
n = 3233,  phi = 60 x 52 = 3120.   gcd(17, 3120) = 1

d = 17^-1 mod 3120 (Extended Euclid):
  q |    r |    s |    t
    | 3120 |    1 |    0
    |   17 |    0 |    1
183 |    9 |    1 | -183
  1 |    8 |   -1 |  184
  1 |    1 |    2 | -367
  8 |    0 |  -17 | 3120
t = -367 -> -367 + 3120 = 2753 = d   (check: 17 x 2753 = 46801 = 15 x 3120 + 1)

Encrypt: C = 65^17 mod 3233
Exponent 17 = 10001 (binary)
bit | square           | multiply by 65 (agar bit=1) | result
 1  | 1^2 mod 3233 = 1      | 1*65 mod 3233 = 65     | 65
 0  | 65^2 mod 3233 = 992    | -                     | 992
 0  | 992^2 mod 3233 = 1232   | -                     | 1232
 0  | 1232^2 mod 3233 = 1547   | -                     | 1547
 1  | 1547^2 mod 3233 = 789    | 789*65 mod 3233 = 2790   | 2790
C = 2790
Decrypt: M = 2790^2753 mod 3233 = 65  (original; same method, 12 bits ka exponent)
```

DHYAN RAKHO

Message M, n se chhota hona chahiye.
φ(n) = (p-1)(q-1) hai, n-1 nahi.

TIP / YAAD RAKHO

Order: **p, q → n → φ → e → d**. Encrypt public key (e) se, decrypt private key (d) se.
Bade power ke liye **square-and-multiply**: har step par mod lo, direct power mat nikalo.
d check: (e x d) mod φ(n) = 1 hona chahiye.

19. Diffie-Hellman Key Exchange

Week 19 | Key agreement

Do parties public channel par **shared secret key** bana lete hain bina secret bheje. Ye **encryption nahi**, sirf key exchange hai. Security: **discrete logarithm problem** (g^a mod p se a nikalna mushkil).

FORMULA

Public: prime **p** aur generator **g** (primitive root of p).
Alice: secret a → $A = g^a \bmod p$ (bheiti hai). Bob: secret b → $B = g^b \bmod p$ (bheita hai).
Shared key: $K = B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$

SOLVED EXAMPLE

Q1. p = 23, g = 5, Alice a = 6, Bob b = 15.

```
Alice: A = 5^6 mod 23
Exponent 6 = 110 (binary)
bit | square           | multiply by 5 (agar bit=1) | result
 1  | 1^2 mod 23 = 1      | 1*5 mod 23 = 5      | 5
 1  | 5^2 mod 23 = 2      | 2*5 mod 23 = 10     | 10
 0  | 10^2 mod 23 = 8      | -                     | 8
      = 8
Bob  : B = 5^15 mod 23 = 19

Alice: K = B^a = 19^6 mod 23  = 2
Bob  : K = A^b = 8^15 mod 23 = 2
Shared key K = 2  (dono taraf same)
```

Q2 (Stallings example). p = 353, g = 3, a = 97, b = 233.

```
A = 3^97  mod 353 = 40
B = 3^233 mod 353 = 248
Alice: K = 248^97 mod 353 = 160
Bob  : K = 40^233 mod 353 = 160
Shared key K = 160
```

Man-in-the-Middle attack: Eve (attacker) Alice ke saath key K1 aur Bob ke saath key K2 bana leti hai. Alice sochti hai wo Bob se baat kar rahi hai, aur Eve dono taraf ka traffic padh/badal sakti hai, kyunki DH me **authentication nahi** hota. Fix: digital signatures / certificates.

TIP / YAAD RAKHO

Shared key ka formula yaad rakho: **$g^{ab} \bmod p$**.
Attacker ke paas p, g, A, B hain par a ya b nahi.

20. Hash Functions: Properties and Applications

Week 20 | Integrity

Hash function **h(M)** kisi bhi length ke message ko **fixed-length digest** me badalta hai. Isme **koi key nahi** hoti aur ye one-way hota hai.

FORMULA

h(M) = fixed-size digest (e.g. SHA-256 → 256 bit)
Properties: (1) Compression + easy to compute. (2) **Preimage resistance** (one-way): h se M nikalna mushkil. (3) **Second-preimage resistance**: M1 diya ho to M2 ≠ M1 with h(M1) = h(M2) dhoondna mushkil. (4) **Collision resistance**: koi bhi M1 ≠ M2 with h(M1) = h(M2) dhoondna mushkil.
Birthday bound: n-bit hash me collision ~ $2^{n/2}$ tries me mil jata hai.

SOLVED EXAMPLE

Q1 (avalanche effect). Message me sirf ek letter badla: abc → abd. SHA-256:

```
SHA-256(abc) = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
SHA-256(abd) = a52d159f262b2c6ddb724a61840befc36eb30c88877a4030b65cbe86298449c9
Alag bits: 122 out of 256  (lagbhag aadhe  yaani poora digest badal gaya)
```

Q2 (kamzor toy hash). h(M) = (ASCII codes ka sum) mod 256.

```
h(AB) = (65 + 66) mod 256 = 131
h(BA) = (66 + 65) mod 256 = 131   -> collision! (AB != BA par same hash)
Isliye ye hash secure nahi: order ka koi asar nahi. Asli hashes me bits mix hote hain (avalanche).
```

TIP / YAAD RAKHO

Applications: message integrity, digital signatures, password storage (hash + salt), MAC/HMAC, file checksums. blockchain.

Hash ek key-less function hai; **MAC** me secret key hoti hai.

21. Secure Hash Algorithm (SHA-1 walkthrough and SHA family)

Week 21 | Cryptographic hash

SHA-1 message ko **512-bit blocks** me process karta hai aur **160-bit digest** deta hai. Merkle-Damgård structure: har block compression function se guzarta hai aur chaining value (H0..H4) update hoti hai.

FORMULA

Padding: message ke end me bit '1', phir '0's jab tak length ≡ 448 (mod 512), phir 64-bit original message length.

Initial H: H0 = 67452301. H1 = EFCDAB89. H2 = 98BADCFE. H3 = 10325476. H4 = C3D2E1F0.

Message schedule: W[0..15] = block ke 16 words; W[t] = ROTL^{1}(W[t-3] ⊕ W[t-8] ⊕ W[t-14] ⊕ W[t-16]) for t = 16..79.

80 steps: temp = ROTL^{5}(a) + f_t(b,c,d) + e + K_t + W_t (mod 2^{32}); e = d; d = c; c = ROTL^{30}(b); b = a; a = temp.

f_t: t 0-19: (b∧c) ∨ (¬b∧d), K = 5A827999; t 20-39: b⊕c⊕d, K = 6ED9EBA1; t 40-59: (b∧c) ∨ (b∧d) ∨ (c∧d), K = 8F1BBCDC: t 60-79: b⊕c⊕d. K = CA62C1D6.

Final: $H_i = H_i + (a, b, c, d, e) \pmod{2^{32}}$. Digest = H0 H1 H2 H3 H4.

SOLVED EXAMPLE

O. SHA-1("abc") nikalo (single block).

Step 1: Padding. "abc" = 61 62 63 (24 bits). Phir 0x80, phir zeros, aakhri 8 bytes me length 0x18.

```
Padded block (64 bytes):
61 62 63 80 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 18
```

Step 2: Message schedule.

```
W0  = 61626380   <- 'abc' + 1 bit padding (0x80)
W1  = 00000000
W2  = 00000000
W3  = 00000000
W4  = 00000000
W5  = 00000000
W6  = 00000000
W7  = 00000000
W8  = 00000000
W9  = 00000000
W10 = 00000000
W11 = 00000000
W12 = 00000000
W13 = 00000000
W14 = 00000000
W15 = 00000018   <- message length = 24 bits (0x18)

W16 = ROTL1(W13 xor W8 xor W2 xor W0) = ROTL1(61626380) = c2c4c700
(W17..W79 isi rule se; kul 80 words)
```

Step 3: Pehle 4 steps (a, b, c, d, e).

```
t | f_t(b,c,d)  | K_t      | W_t      |  a        b        c        d        e
- |              |          |          |  67452301 efcdab89 98badcfe 10325476 c3d2e1f0
0 | 98badcfe     | 5a827999 | 61626380 |  0116fc33 67452301 7bf36ae2 98badcfe 10325476
1 | fbfbfefe     | 5a827999 | 00000000 |  8990536d 0116fc33 59d148c0 7bf36ae2 98badcfe
```

```
2 | 7bf14ac0      | 5a827999 | 00000000 |  a1390f08 8990536d c045bf0c 59d148c0 7bf36ae2
3 | d0411b8c      | 5a827999 | 00000000 |  cdd8e11b a1390f08 626414db c045bf0c 59d148c0
...
79 | after 80 steps: a=42541b35 b=5738d5e1 c=21834873 d=681e6df6 e=d8fdf6ad
```

Step 4: Final addition.

```
H0 = 67452301 + a = a9993e36
H1 = EFCDAB89 + b = 4706816a
H2 = 98BADCFE + c = ba3e2571
H3 = 10325476 + d = 7850c26c
H4 = C3D2E1F0 + e = 9cd0d89d

SHA-1(abc) = a9993e364706816aba3e25717850c26c9cd0d89d
SHA-256(abc) =
  ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
(same idea, 64 steps, 256-bit)
```

TIP / YAAD RAKHO

Yaad rakho: SHA-1: **160 digest. 512 block. 80 rounds**. SHA-256: 256/512/64. SHA-512: 512/1024/80. SHA-1 ab collision-resistant nahi mana jata; SHA-256/SHA-3 recommended.
Exam me aksar padding aur parameters (digest, block, rounds) poochhe jate hain.

SHA family (parameters table, exam me aksar poochha jata hai)

Algorithm	Digest	Block	Word	Rounds	Max message
SHA-1	160 bit	512 bit	32 bit	80	2^64 - 1
SHA-224	224 bit	512 bit	32 bit	64	2^64 - 1
SHA-256	256 bit	512 bit	32 bit	64	2^64 - 1
SHA-384	384 bit	1024 bit	64 bit	80	2^128 - 1
SHA-512	512 bit	1024 bit	64 bit	80	2^128 - 1

22. HMAC (Hash-based Message Authentication Code)

Week 22 | MAC

MAC message ke saath ek tag bhejta hai jo **secret key** se banta hai: receiver tag verify karke **integrity + authenticity** check karta hai. HMAC ye kaam hash function (SHA-256 etc.) se karta hai.

FORMULA

HMAC(K, M) = H((K' ⊕ opad) || H((K' ⊕ ipad) || M))
K' = key ko hash block size B tak zero-pad (key B se lambi ho to pehle hash karo). B = 64 bytes (SHA-1/SHA-256), 128 bytes (SHA-512).
ipad = byte 0x36 repeat B baar, opad = byte 0x5C repeat B baar. || = concatenation.

SOLVED EXAMPLE

Q. HMAC-SHA-256, key = 20 bytes of 0x0b, message = **"Hi There"** (RFC 4231 test case 1).

```
Step 1: K' = 0b x 20 bytes + 00 x 44 bytes  (kul 64 bytes)
Step 2: K' xor ipad = 3d x 20 bytes + 36 x 44 bytes   (0b xor 36 = 3d)
Step 3: K' xor opad = 57 x 20 bytes + 5c x 44 bytes   (0b xor 5c = 57)

Step 4: inner = SHA-256( (K' xor ipad) || "Hi There" )
      = 92ab4d9a1f3b6152bca9dd9e69af43f4ce99e42fd4e30ff972c48025b9f9cfef
Step 5: outer = SHA-256( (K' xor opad) || inner )
      = b0344c61d8db38535ca8afceaf0bf12b881dc200c9833da726e9376c2e32cff7

HMAC = b0344c61d8db38535ca8afceaf0bf12b881dc200c9833da726e9376c2e32cff7
(Python hmac module se verify: same output)
```

TIP / YAAD RAKHO

Formula ko andar se padho: pehle **inner hash** (ipad ke saath), phir **outer hash** (opad ke saath). HMAC simple H(K || M) se behtar hai kyunki wo **length-extension attack** se bachata hai.
Tag same aaye to message sahi aur sender ke paas key hai.

APPENDIX: Tables (DES aur AES)

Ye wohi tables hain jo upar ke examples me use hui hain (test vectors se verify). Exam me table di jati hai to is se match karke check kar lo.

IP (Initial Permutation)

```
58 50 42 34 26 18 10  2
60 52 44 36 28 20 12  4
62 54 46 38 30 22 14  6
64 56 48 40 32 24 16  8
57 49 41 33 25 17  9  1
59 51 43 35 27 19 11  3
61 53 45 37 29 21 13  5
63 55 47 39 31 23 15  7
```

FP / IP^-1 (Final Permutation)

```
40  8 48 16 56 24 64 32
39  7 47 15 55 23 63 31
38  6 46 14 54 22 62 30
37  5 45 13 53 21 61 29
36  4 44 12 52 20 60 28
35  3 43 11 51 19 59 27
34  2 42 10 50 18 58 26
33  1 41  9 49 17 57 25
```

E (Expansion, 32 to 48)

```
32  1  2  3  4  5
 4  5  6  7  8  9
 8  9 10 11 12 13
12 13 14 15 16 17
16 17 18 19 20 21
20 21 22 23 24 25
24 25 26 27 28 29
28 29 30 31 32  1
```

P (Permutation, 32 bit)

```
16  7 20 21 29 12 28 17
 1 15 23 26  5 18 31 10
 2  8 24 14 32 27  3  9
19 13 30  6 22 11  4 25
```

PC-1 (64 to 56)

```
57 49 41 33 25 17  9
 1 58 50 42 34 26 18
10  2 59 51 43 35 27
19 11  3 60 52 44 36
63 55 47 39 31 23 15
 7 62 54 46 38 30 22
14  6 61 53 45 37 29
21 13  5 28 20 12  4
```

PC-2 (56 to 48)

```
14 17 11 24  1  5
 3 28 15  6 21 10
23 19 12  4 26  8
16  7 27 20 13  2
41 52 31 37 47 55
30 40 51 45 33 48
44 49 39 56 34 53
46 42 50 36 29 32
```

DES key shifts

```
Shifts per round 1..16: 1 1 2 2 2 2 2 2 1 2 2 2 2 2 2 1
```

DES S-boxes S1 to S8

```
S1:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0: 14  4 13  1  2 15 11  8  3 10  6 12  5  9  0  7
  row 1:  0 15  7  4 14  2 13  1 10  6 12 11  9  5  3  8
  row 2:  4  1 14  8 13  6  2 11 15 12  9  7  3 10  5  0
  row 3: 15 12  8  2  4  9  1  7  5 11  3 14 10  0  6 13

S2:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0: 15  1  8 14  6 11  3  4  9  7  2 13 12  0  5 10
  row 1:  3 13  4  7 15  2  8 14 12  0  1 10  6  9 11  5
  row 2:  0 14  7 11 10  4 13  1  5  8 12  6  9  3  2 15
  row 3: 13  8 10  1  3 15  4  2 11  6  7 12  0  5 14  9

S3:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0: 10  0  9 14  6  3 15  5  1 13 12  7 11  4  2  8
  row 1: 13  7  0  9  3  4  6 10  2  8  5 14 12 11 15  1
  row 2: 13  6  4  9  8 15  3  0 11  1  2 12  5 10 14  7
  row 3:  1 10 13  0  6  9  8  7  4 15 14  3 11  5  2 12

S4:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0:  7 13 14  3  0  6  9 10  1  2  8  5 11 12  4 15
  row 1: 13  8 11  5  6 15  0  3  4  7  2 12  1 10 14  9
  row 2: 10  6  9  0 12 11  7 13 15  1  3 14  5  2  8  4
  row 3:  3 15  0  6 10  1 13  8  9  4  5 11 12  7  2 14

S5:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0:  2 12  4  1  7 10 11  6  8  5  3 15 13  0 14  9
  row 1: 14 11  2 12  4  7 13  1  5  0 15 10  3  9  8  6
  row 2:  4  2  1 11 10 13  7  8 15  9 12  5  6  3  0 14
  row 3: 11  8 12  7  1 14  2 13  6 15  0  9 10  4  5  3

S6:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0: 12  1 10 15  9  2  6  8  0 13  3  4 14  7  5 11
  row 1: 10 15  4  2  7 12  9  5  6  1 13 14  0 11  3  8
  row 2:  9 14 15  5  2  8 12  3  7  0  4 10  1 13 11  6
  row 3:  4  3  2 12  9  5 15 10 11 14  1  7  6  0  8 13

S7:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0:  4 11  2 14 15  0  8 13  3 12  9  7  5 10  6  1
  row 1: 13  0 11  7  4  9  1 10 14  3  5 12  2 15  8  6
  row 2:  1  4 11 13 12  3  7 14 10 15  6  8  0  5  9  2
  row 3:  6 11 13  8  1  4 10  7  9  5  0 15 14  2  3 12

S8:      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  row 0: 13  2  8  4  6 15 11  1 10  9  3 14  5  0 12  7
  row 1:  1 15 13  8 10  3  7  4 12  5  6 11  0 14  9  2
  row 2:  7 11  4  1  9 12 14  2  0  6 10 13 15  3  5  8
  row 3:  2  1 14  7  4 10  8 13 15 12  9  0  3  5  6 11
```

AES S-box (row = high nibble, column = low nibble)

```
      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
  0x: 63 7c 77 7b f2 6b 6f c5 30 01 67 2b fe d7 ab 76
  1x: ca 82 c9 7d fa 59 47 f0 ad d4 a2 af 9c a4 72 c0
  2x: b7 fd 93 26 36 3f f7 cc 34 a5 e5 f1 71 d8 31 15
  3x: 04 c7 23 c3 18 96 05 9a 07 12 80 e2 eb 27 b2 75
  4x: 09 83 2c 1a 1b 6e 5a a0 52 3b d6 b3 29 e3 2f 84
  5x: 53 d1 00 ed 20 fc b1 5b 6a cb be 39 4a 4c 58 cf
  6x: d0 ef aa fb 43 4d 33 85 45 f9 02 7f 50 3c 9f a8
  7x: 51 a3 40 8f 92 9d 38 f5 bc b6 da 21 10 ff f3 d2
  8x: cd 0c 13 ec 5f 97 44 17 c4 a7 7e 3d 64 5d 19 73
  9x: 60 81 4f dc 22 2a 90 88 46 ee b8 14 de 5e 0b db
  ax: e0 32 3a 0a 49 06 24 5c c2 d3 ac 62 91 95 e4 79
  bx: e7 c8 37 6d 8d d5 4e a9 6c 56 f4 ea 65 7a ae 08
  cx: ba 78 25 2e 1c a6 b4 c6 e8 dd 74 1f 4b bd 8b 8a
  dx: 70 3e b5 66 48 03 f6 0e 61 35 57 b9 86 c1 1d 9e
  ex: e1 f8 98 11 69 d9 8e 94 9b 1e 87 e9 ce 55 28 df
  fx: 8c a1 89 0d bf e6 42 68 41 99 2d 0f b0 54 bb 16

Rcon[1..10] = 01 02 04 08 10 20 40 80 1b 36
```